00 Pedagogical Foundation

Dylon La Rue

Before We Begin

Alas, a book that can be judged by its cover!

The goal here is to introduce you, my dear reader, to the actual mathematics underlying consciousness research — through a series of counting games. There isn't much mathematical maturity expected here, but I am expecting a sufficiently weighted balance of curiosity, perseverance, and a capacity to reason at least around the high school level. Don't be afraid to use a search engine, reach out to a friend, ask a teacher, or heaven forbid go to a library if you get stuck; research skills are as much, if not more, important than any other skill in the mathematician's toolbox. Seriously, look stuff up.

Something important to explicitly discuss up front is the importance of **letters** in mathematics. Using placeholders like letters instead of numbers lets us focus on the logic surrounding the specific values we can operate on. I'll rely heavily on the use of letters in this text. Number theory and combinatorics texts tend to use k, n, m, p; when thinking in geometric dimensions, l, w, h or s; when thinking sequentially, i for indexing. You get the gist.

Throughout this chapter, every concept we build by hand will also exist as a runnable Python module in the companion code `principia/foundations/`. By the end, you will have built a small software library that encodes the same mathematics the rest of the book extends into uncharted territory.

1 Sets, Logic, and Your First Database

A **set** is a well-defined, unordered collection of objects called its **elements**. The key word is *well-defined*: given any object, we must be able to say definitively whether it belongs to the set or not. A "collection of cool numbers" is not a set. The collection of prime numbers less than 20 is.

We write sets with curly braces. If

$$D = \{1, 4, 9, 16, 25\},$$

then D is the set of perfect squares from 1 to 25. If

$$A = \{a, b, c\},$$

then A is a set of three letters. Sets don't care about order: $\{a, b, c\} = \{c, a, b\}$.

Membership. The symbol $\in$ means "is an element of." It looks like an E or ϵ. The statement "negative sixty-seven is a member of the set of integers" can be written set-theoretically as $-67 \in \mathbb{Z}$.

Subsets and cardinality. The **subset** symbol $\subseteq$ says one set lives entirely inside another. "The natural numbers are a subset of the integers" is $\mathbb{N} \subseteq \mathbb{Z}$. Note that a set is a subset of itself. The **cardinality** of a set is its number of elements, written $|D| = 5$ and $|A| = 3$. If $S \subseteq U$ and $|S| < |U|$, we say S is a **proper subset** of U and write $S \subset U$.

Power sets. A special case: the set of *all* subsets of a set X is the **power set** of X, denoted $\mathscr{P}(X)$. For $X = \{a, b, c\}$:

$$\mathscr{P}(X) = \big\{\varnothing, \{a\}, \{b\}, \{c\}, \{a, b\}, \{a, c\}, \{b, c\}, \{a, b, c\}\big\}.$$

One thing you can convince yourself by ordering a power set like we have above is that for every subset S which exists as a member of X, there exists another subset of X whose elements are exactly all of those in X but *not* in S. We call this the **complement** of S and denote it $\overline{S}$:

$$\overline{S} = \{x \,:\, x \in X \wedge x \notin S\}.$$

Symbolic logic. Since sets are studied from a mathematical point of view, it's customary to use mathematical symbols in the **statements** we construct about sets. Alongside mathematics, **symbolic logic** is a key subject from which we adopt many of our hieroglyphs. Symbolic logic systems encode statements and propositions into variables (it's customary to use p, q, and r, but any agreed-upon symbol works) and express logical operations using symbols.

The **universal quantifier**, $\forall$, is shorthand for "for all" or "for everything of the form." Our second quantifier, the **existential quantifier**, $\exists$, means "there exists." We can now write statements like "every natural number n has a square m that is also natural" as $\forall n \in \mathbb{N},\ \exists m \in \mathbb{N} \,:\, m = n^2$.

Equally important are **logical connectives**. Connectives help us use "if-then," "and," "or," and "not" to determine the truth of mathematical and logical statements:

- $p \Rightarrow q$: "p implies q" (if-then)
- $r \wedge s$: "r and s" (conjunction)
- $p \vee s$: "p or s" (disjunction)
- $\neg q$ or $\sim q$: "not q" (negation)

The PostgreSQL parallel. If you have ever seen a spreadsheet, you already have a working mental model of a **relational database**. The mathematical foundations of relational databases *are* set theory, formalized by Edgar Codd in 1970 and executed today by engines like **PostgreSQL 17**. A database table is a set of rows; each row is an element. The SQL language gives us the same operations we just learned, in a syntax designed for data:

Set Theory	SQL (PostgreSQL 17)
Set S	`TABLE S` or `VIEW S`
$x \in S$	`SELECT * FROM S WHERE x = ...`
$\lvert S \rvert$	`SELECT COUNT(*) FROM S`
$A \cup B$	`SELECT * FROM A UNION SELECT * FROM B`
$A \cap B$	`SELECT * FROM A INTERSECT SELECT * FROM B`
$A - B$	`SELECT * FROM A EXCEPT SELECT * FROM B`
$\forall$	`ALL`
$\exists$	`EXISTS`

This is not a metaphor. When you write SQL, you are literally writing set theory in a slightly different alphabet. For example, to find the intersection of two tables:

```
-- PostgreSQL 17: Set Intersection
SELECT id FROM set_A
INTERSECT
SELECT id FROM set_B;
```

The companion module `principia.foundations.sets` implements a `FiniteSet` class in Python that mirrors both the mathematical definitions and the SQL operations:

```
from principia.foundations.sets import FiniteSet

D = FiniteSet({1, 4, 9, 16, 25})
A = FiniteSet({'a', 'b', 'c'})

print(9 in D)              # True  (membership)
print(len(D))              # 5     (cardinality)
print(D.power_set())       # all 32 subsets of D
print(A.complement({'a'})) # {'b', 'c'}
```

2 Set Operations and Indexing

The big three set operations we'll worry about are **union**, **difference**, and **intersection**.

I like to think of unions as a kind of careful addition. Unions help us determine objects that are elements of any one of a number of given sets. The symbol for union looks like a big "u", $\cup$. The union is defined as

$$X \cup Y = \{x : x \in X \vee x \in Y\}.$$

The **difference** between sets is given by the minus sign:

$$U - S = \{x : x \in U \wedge x \notin S\}.$$

Intersections are denoted with the union symbol upside down:

$$X \cap Y = \{x : x \in X \wedge x \in Y\}.$$

Venn diagrams were introduced by mathematician John Venn in 1880 in the first article of *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science.* An early quote of Venn's reads:

> The great bulk of the propositions which we commonly meet with are founded, and rightly founded, on an imperfect knowledge of the actual mutual relations of the implied classes to one another.

This is an insight that will echo through the entire book: the structures we study are often discovered through *imperfect knowledge of mutual relations* — what we will later call the Topological Phase Transition invariant.

Indexed operations. When we have a whole family of sets $A_1, A_2, \dots, A_n$, we can express their union and intersection compactly:

$$\bigcup_{i=1}^{n} A_i = A_1 \cup A_2 \cup \cdots \cup A_n, \qquad \bigcap_{i=1}^{n} A_i = A_1 \cap A_2 \cap \cdots \cap A_n.$$

This indexed notation will reappear throughout the book, particularly when we partition the multiplicative group $\mathbb{F}_p^*$ into cosets of its subgroups.

3 Relations, Functions, and Mapping

One set operation we seemingly skipped lightly over earlier is the **Cartesian product**, named after 17th century French polymath René Descartes. Descartes was a pioneer in connecting geometry to algebra; he allowed line segments of length one and demonstrated how to find monstrous polynomials in the analytic study of curved lines.

An n-ary Cartesian product is the set of length-n lists that can be made from n sets, with the i-th element of a given list coming from the i-th set. If $A = \{a_1, a_2, \dots, a_n\}$ and $B = \{b_1, b_2, \dots, b_m\}$, then

$$A \times B = \{(a, b) \,:\, a \in A,\ b \in B\}.$$

A **relation** is a subset of the Cartesian product of sets. A **function** $f : A \to B$ is a special kind of relation: for every element $a \in A$, there exists exactly one element $b \in B$ such that (a, b) is in the relation. We write $f(a) = b$ and call A the **domain** and B the **codomain**.

Three important properties of functions:

- **Injective** (one-to-one): different inputs give different outputs.
- **Surjective** (onto): every element of B is hit by some input.
- **Bijective**: both injective and surjective — a perfect pairing.

The SQL parallel. The Cartesian product is exactly what happens when you `JOIN` two tables without a condition: every row of table A is paired with every row of table B. Adding a `WHERE` clause filters this product down to a *relation.* Adding a `UNIQUE` constraint or a `PRIMARY KEY` on the joining column enforces injectivity.

The Psycopg3 parallel: Functors. Functions map elements between sets. But what happens when we need to map entire *systems* of sets to another system while preserving their structure? This is a **Functor**, the core object of Category Theory.

When you use the **psycopg3** library in Python to connect to PostgreSQL, you are explicitly building a Functor. It takes the Category of Relational Data (PostgreSQL types like `uuid`, `jsonb`, `numeric`) and maps it flawlessly into the Category of Object-Oriented State (Python types like `UUID`, `dict`, `Decimal`).

To make this rigorous, we have provided a companion verification script `scripts/psycopg_functorial_b` that extends this functor into the Adelic Ring of the pAQFT model:

```
import psycopg
from scripts.psycopg_functorial_bridge import PsycopgFunctor, LocallyCovariantFunctor

# 1. Functor mapping the Postgres Category to the Python Category
py_obj = PsycopgFunctor.map_record(pg_record)
print(type(py_obj.payload)) # <class 'dict'>

# 2. Functor mapping the Python Category into the Adelic Ring (Golden Fields)
adelic_state = LocallyCovariantFunctor.lift(py_obj, prime=229)
print(adelic_state.residue) # 15 (mapped cleanly into F_229)
```

The structure is preserved across the boundary. This Functorial bridge is how we will eventually build the arithmetic scheme Algebra: by mapping geometric shapes into number-theoretic primes without breaking their internal logic.

4 Sequences, Sums, and Series

Oh my! You, my dear reader, are already intuitively familiar with at least one of these concepts, but by the end of this journey you will be intimately familiar with all three and the roles they play in the grand theatre of discrete mathematics. If you've taken calculus II, you should be familiar with how important they are for communicating and solving problems on the continuous side of the mathematical coin.

A **sequence** is an infinite list $a_1, a_2, a_3, a_4, \ldots$, usually denoted by mathematicians with the notation $\{a_s\}$, where position is given by the index s. A list is a collection of objects in which repetition is allowed and order defines unique identity.

A **sum** is simply an addition; more fondly, a sum is the first combinatorial step taken past counting on our fingers. If you leave here with any one idea lodged in between your ear-holes, I hope that idea is that there are seemingly countless counting techniques.

A **series** is then the love-child of the preceding two: an infinite sum. I will often use the **sigma-summation notation**, which uses the upper-case Greek letter sigma, Σ, and indices like we saw with indexed set operations. We use a variable set equal to the sum's lower bound,

and write the upper bound on top:

$$\sum_{x=1}^{n} a_x = a_1 + a_2 + a_3 + \cdots + a_n.$$

The Python parallel. Sequences are generators. Series are `sum()` applied to generators.

```
# The sequence of squares: 1, 4, 9, 16, ...
def squares():
    n = 1
    while True:
        yield n * n
        n += 1

# Partial sum of the first 10 squares
from itertools import islice
print(sum(islice(squares(), 10)))  # 385
```

The SQL parallel. Summation is aggregation: `SELECT SUM(value) FROM sequence_table WHERE index <= 10`. Window functions let you compute running partial sums: `SUM(value) OVER (ORDER BY index)`.

5 Figurate Numbers and Dot Lattices

All that work we've just done was a student-led introduction to some of the publicly lesser-known tools and definitions of those foolish mathematicians, as well as the construction of our first "stage," or playing field. But who's going to act in our plays written in logic on a paper-thin stage? What games can be played and who plays them, and in what ways? Honestly, if you can describe a thing, you can write math about or with it; sometimes that description *is* the math that we want to do about our given thing.

So that I may make clear both the beauty and power of counting, let us count some shapes.

5.1 Triangular Numbers

The triangular numbers T_n count the number of objects it takes to form an equilateral triangle of side-length n. As we increment n by 1, the number we add to the sequence also increases by 1:

$$\begin{array}{rcl} n & = & 1,\ 2,\ 3,\ 4,\ 5,\ 6,\ldots \\ T_n & = & 1,\ 3,\ 6,\ 10,\ 15,\ 21,\ldots \end{array}$$

The apparent pattern is $T_n = \frac{n(n+1)}{2}$. We'll find this to be the case by *pondering the triangle*. Recall from geometry that the area of a triangle is given by $\frac{1}{2} \cdot \text{base} \cdot \text{height}$, which we can prove by copying a given triangle and creating a rectangle from two copies.

If we take a dot-triangle of side n and place a rotated copy against it, we appear to have a square of side n. But notice how the dot-lattice is *disturbed*: there's an extra pair of sides born from the one-to-one side alignment. We fix this by sliding one of the copies a unit over, producing a rectangle of dimensions $n \times (n+1)$ rather than a square of side n.

Since we used two copies of our triangle to build this rectangle:

$$2T_n = n(n+1) \quad \Longrightarrow \quad T_n = \frac{n(n+1)}{2}.$$

We can also verify this by noting $T_n = 1 + 2 + 3 + \cdots + n$. Say $S = 1 + 2 + 3 + \cdots + (n-1) + n$. If we throw a duplicate in reverse order underneath:

$$\begin{aligned} S &= 1 + 2 + 3 + \cdots + (n-1) + n \\ S &= n + (n-1) + (n-2) + \cdots + 2 + 1 \end{aligned}$$

Now we have two S's on the left-hand side and n many pairs of $(n+1)$:

$$2S = n(n+1), \quad \text{so} \quad S = \frac{n(n+1)}{2}. \quad \square$$

Therefore:

$$T_n = \sum_{x=0}^{n} x = \frac{n(n+1)}{2}.$$

5.2 Square Numbers

The next shape we can build should be very familiar: the **squares**.

The square of the n-th number is denoted n^2 and given by $n \cdot n$. This follows by the definition of a square as a special rectangle whose width equals its height.

How does a square grow? Look at the dot in the opposite corner from our "seed square" (the single dot at position 1). No matter the size of n or n^2, this corner dot is an element of *both* new sides we've added. To fix the double-counting, we subtract it once from the twice we've added it to the square. Thus each new square is given by adding the function $2n - 1$ — the n-th odd number, counting from 1:

$$n^2 = \sum_{k=1}^{n} (2k-1) = 1 + 3 + 5 + 7 + \cdots + (2n-1).$$

This gnomon structure — the L-shaped border added at each step — is one of the oldest ideas in mathematics, dating back to the Pythagoreans. It gives us our first **figurate identity**:

$$T_n + T_{n-1} = n^2.$$

To see this, observe that $T_n = \frac{n(n+1)}{2}$ and $T_{n-1} = \frac{(n-1)n}{2}$, so their sum is $\frac{n(n+1)+n(n-1)}{2} = \frac{n \cdot 2n}{2} = n^2$. But you can also see it with dots: rotate a triangle of side $n-1$ and nestle it into a triangle of side n. They tile a square of side n perfectly.

5.3 More Figurate Flavors

The method generalizes. Pentagonal numbers, hexagonal numbers, and higher polygonal numbers all follow the same pattern: fix a vertex, add a border (gnomon) of the appropriate shape at each step. The r-gonal number of rank n is:

$$P(r, n) = \frac{n\big[(r-2)n - (r-4)\big]}{2}.$$

For $r = 3$ this gives our triangular numbers, for $r = 4$ the squares. The companion module `principia.foundations.figurate` computes all of these and prints dot diagrams to the terminal:

```
from principia.foundations.figurate import triangular, square, polygonal

print([triangular(n) for n in range(1, 8)])
# [1, 3, 6, 10, 15, 21, 28]

print([square(n) for n in range(1, 8)])
# [1, 4, 9, 16, 25, 36, 49]

# Verify the figurate identity
for n in range(1, 20):
    assert triangular(n) + triangular(n-1) == square(n)
```

6 From Counting to Congruence

We have been working with sets, functions, and sequences over the natural numbers $\mathbb{N}$ and integers $\mathbb{Z}$. These are infinite sets. The rest of this book works with a special kind of set that is *finite* — and precisely because it is finite, it has structure that infinite sets lack.

6.1 The von Neumann Construction

Hungarian mathematician, physicist, and the computer scientist who designed modern computer architecture, János "Johnny" von Neumann submitted a paper in his teen years with the following construction of $\mathbb{N}$ as the first step in his construction of the set of transfinite ordinals. We won't need the transfinite ones — $\mathbb{N}$ is more than enough firepower for now.

Johnny the Martian (read Labatut's *The MANIAC*), like any good computer scientist, starts off his set of counting numbers with zero, represented by the empty set $0 = \{\}$. The successor function is defined as $n + 1 = n \cup \{n\}$. Since the empty set has no elements to union with $\{0\}$:

$$\begin{aligned} 1 &= \{0\} = \{\varnothing\}, \\ 2 &= 1 \cup \{1\} = \{0, \{0\}\}, \\ 3 &= 2 \cup \{2\} = \{0, \{0\}, \{0, \{0\}\}\}. \end{aligned}$$

By building $\mathbb{N}$ this way, a given natural number's cardinality is equal to that very number: $|3| = 3$. The construction bootstraps counting from *nothing* — from the empty set — using only the tools we've already learned (sets, union, cardinality). This is the logical foundation that the rest of mathematics stands on.

6.2 Divisibility and Primes

An integer a **divides** b (written $a \mid b$) if there exists an integer k such that $b = ak$. A natural number $p > 1$ is **prime** if its only divisors are 1 and itself. The first few primes are $2, 3, 5, 7, 11, 13, 17, 19, 23, \ldots$

Euclid proved around 300 BCE that there are infinitely many primes. The proof is elegant: assume there are finitely many primes $p_1, p_2, \ldots, p_n$. Form the number $N = p_1 p_2 \cdots p_n + 1$. Then N is not divisible by any p_i (it leaves remainder 1), so either N is prime or it has a prime factor not in our list. Either way, the assumption was wrong. $\square$

6.3 Modular Arithmetic: Clock Math

Here is the key idea that connects everything we've done to the rest of this book.

Modular arithmetic is arithmetic where numbers "wrap around" after reaching a fixed value called the **modulus**. You already know this: a clock wraps around after 12. We write $a \equiv b \pmod{n}$ to mean "a and b have the same remainder when divided by n," or equivalently, $n \mid (a - b)$.

The set of remainders modulo n is denoted $\mathbb{Z}/n\mathbb{Z}$ (or just $\mathbb{Z}_n$ informally). For $n = 5$:

$$\mathbb{Z}/5\mathbb{Z} = \{0, 1, 2, 3, 4\}.$$

Addition and multiplication in this set always wrap around:

$$3 + 4 \equiv 2 \pmod{5}, \qquad 3 \cdot 4 \equiv 2 \pmod{5}.$$

When the modulus p is **prime**, something remarkable happens: every nonzero element has a multiplicative inverse. The set $\{0, 1, 2, \ldots, p-1\}$ with addition and multiplication modulo p forms a **finite field**, denoted $\mathbb{F}_p$. This is the playing field for the entire book.

```
from principia.foundations.modular import ModularInt

# Working in F_7
a = ModularInt(3, 7)
b = ModularInt(5, 7)
print(a + b)    # 1 (mod 7)
print(a * b)    # 1 (mod 7)  --- 3 and 5 are inverses!
print(a ** 6)   # 1 (mod 7)  --- Fermat's Little Theorem
```

6.4 The Bridge

We have now assembled every tool we need.

From sets, we learned how to define collections and reason about membership. From functions, we learned how to map one structure to another. From figurate numbers, we learned that shapes encode arithmetic identities. From modular arithmetic, we learned that finite worlds have algebraic structure that infinite worlds lack.

The next chapter asks a single question that will occupy us for the rest of this book:

$$\textit{Does the polynomial } X^2 - X - 1 = 0 \textit{ have roots in } \mathbb{F}_p\textit{?}$$

This is the polynomial whose real root is the golden ratio, $\varphi = \frac{1+\sqrt{5}}{2}$. In finite fields, this question turns out to depend entirely on whether 5 is a square modulo p — a question about the *structure* of primes that connects number theory, algebra, and geometry in ways that no one expected when Fibonacci first wrote down his famous sequence in 1202.

Welcome to the Arithmetic Scheme Topos.

Reader Exercises

1. Draw dot diagrams for T_3, T_4, and T_5.
2. Draw dot diagrams for 3^2, 4^2, and 5^2.
3. Use dots to show that $T_n + T_{n-1} = n^2$ for $n = 4$.
4. Verify that $T_{10} = 55$ using both the formula and the Gauss pairing trick ($S = 1+2+\cdots+10$, reverse and add).
5. In $\mathbb{F}_7$: find the multiplicative inverse of every nonzero element. (Hint: for each $a \in \{1, 2, 3, 4, 5, 6\}$, find b such that $ab \equiv 1 \pmod 7$.)
6. Write a Python function that generates the first n pentagonal numbers using the formula $P(5, k) = \frac{k(3k-1)}{2}$.
7. **Challenge**: Using the companion code, find all primes $p < 100$ for which $X^2 - X - 1 = 0$ has roots in $\mathbb{F}_p$. What pattern do you notice about these primes modulo 5?

Companion Code: The Psycopg Functorial Bridge

```
#!/usr/bin/env python3
"""
Psycopg3 Functorial Bridge: Category Theory in Practice

This script demonstrates the "psycopg3 parallel" from Principia Psychedelia
```

```
Vol I, Chapter 0. It serves as a Rosetta Stone for understanding Category Theory
Functors using standard data engineering concepts.

To understand Category Theory, you only need to know three things:
1. Objects (The nouns: Sets, Types, or SQL Tables)
2. Morphisms (The verbs/arrows: Functions, Pointers, or SQL Foreign Keys)
3. Functors (The translators: Maps that convert Objects to Objects, and
   Morphisms to Morphisms, while PRESERVING the exact structural relationship).

Usage: python3 scripts/psycopg_functorial_bridge.py
"""

from dataclasses import dataclass
from typing import Dict, Any, TypeVar, Generic, Optional
import json

# ---------------------------------------------------------------------
# CATEGORY 1: PostgreSQL (The Category of Relational Data)
# ---------------------------------------------------------------------
# In this category:
# - An OBJECT is a Table (a set of rows).
# - A MORPHISM is a Foreign Key (an arrow pointing from one table to another).

@dataclass
class PostgresParentRow:
    """An Object in the Postgres Category (e.g. the 'parents' table)."""
    id: int
    data: str # JSONB

@dataclass
class PostgresChildRow:
    """An Object in the Postgres Category (e.g. the 'children' table)."""
    id: int
   parent_id: int  # This is the MORPHISM (Foreign Key) pointing to PostgresParentRow!
    numeric_value: float

class PostgresCategory:
    """Simulates a database engine."""
    @staticmethod
    def query_records():
        # Simulated database state
        parent = PostgresParentRow(id=1, data='{"state": "stable"}')
        child = PostgresChildRow(id=229, parent_id=1, numeric_value=1.618)
        return parent, child
```

```
# ------------------------------------------------------------------
# CATEGORY 2: Python (The Category of Object-Oriented State)
# ------------------------------------------------------------------
# In this category:
# - An OBJECT is a Class Instance (or a primitive like dict/float).
# - A MORPHISM is a Memory Reference (a variable pointing to another instance).

@dataclass
class PythonParent:
    """An Object in the Python Category."""
    uuid: int
    payload: dict

@dataclass
class PythonChild:
    """An Object in the Python Category."""
    uuid: int
   parent_ref: PythonParent  # This is the MORPHISM (Memory Reference) pointing to PythonPare
    golden_approximation: float

# ------------------------------------------------------------------
# THE FUNCTOR: Psycopg3
# ------------------------------------------------------------------
# A Functor maps Category 1 to Category 2.
# It MUST preserve the relationships (the Morphisms).

class PsycopgFunctor:
    """
    The Functor: PostgresCategory -> PythonCategory
    Notice how it translates the objects (Row -> Class) AND the morphism
    (Foreign Key -> Memory Reference) flawlessly, preserving the structure!
    """
    @staticmethod
    def map_parent(row: PostgresParentRow) -> PythonParent:
        # Maps SQL JSONB to Python dict
        return PythonParent(uuid=row.id, payload=json.loads(row.data))

    @staticmethod
   def map_child(child_row: PostgresChildRow, parent_row: PostgresParentRow) -> PythonChild
        # First, map the parent object
        py_parent = PsycopgFunctor.map_parent(parent_row)

        # Then, map the child object, translating the Foreign Key (parent_id)
        # directly into a Python memory reference (py_parent)
        return PythonChild(
```

```
            uuid=child_row.id,
            parent_ref=py_parent, # Structure Preserved! The Functor works.
            golden_approximation=child_row.numeric_value
        )

# ---------------------------------------------------------------------
# CATEGORY 3: The Golden Fields (Adelic Ring pAQFT)
# ---------------------------------------------------------------------

@dataclass
class AdelicState:
    """An Object in the Golden Field F_p."""
    p: int
    residue: int

class LocallyCovariantFunctor:
    """
    The pAQFT Functor: PythonCategory -> AdelicCategory
    Maps floating point approximations into exact discrete moduli.
    """
    @staticmethod
    def lift(obj: PythonChild, prime: int) -> AdelicState:
      # Scale the approximation by 1000 to capture decimals and reduce modulo p
        scaled = int(obj.golden_approximation * 1000)
        return AdelicState(p=prime, residue=scaled % prime)

# ---------------------------------------------------------------------
# THE BRIDGE EXECUTION
# ---------------------------------------------------------------------

def run_functorial_bridge():
    print("=== The Psycopg3 Functorial Bridge ===")

    # 1. Start in Postgres (Category 1)
    pg_parent, pg_child = PostgresCategory.query_records()
    print(f"\n[PostgreSQL Category]")
    print(f"Parent Object: {pg_parent}")
   print(f"Child Object:  {pg_child} (Morphism: parent_id={pg_child.parent_id})")

    # 2. Functor to Python (Category 2)
    py_child = PsycopgFunctor.map_child(pg_child, pg_parent)
    print(f"\n[Python Category]")
   print(f"Child Object:  uuid={py_child.uuid}, approx={py_child.golden_approximation}")
   print(f"Morphism Preserved! parent_ref points to -> {py_child.parent_ref}")
```

```
    # 3. Functor to Adelic Ring (Category 3)
    adelic_229 = LocallyCovariantFunctor.lift(py_child, 229)
    print(f"\n[Adelic Category (F_229)]")
    print(f"State: {adelic_229}")

    adelic_181 = LocallyCovariantFunctor.lift(py_child, 181)
    print(f"\n[Adelic Category (F_181)]")
    print(f"State: {adelic_181}")

if __name__ == "__main__":
    run_functorial_bridge()
```